Fitra

Design Report on Software Maintainability

Version 1.1

By: Gary Quah, Matthew Wong, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi

Document Change Record

Revision	Description of Change	Approved by	Date
0.10	Initial Template	Gary	15/10/2026
0.20	Edits to template	Gary	21/10/2026
0.30	Edits to template	Gary	25/10/2026
1.00	Final revised edition	Gary	28/10/2026
1.1	Clarified in-process readiness/Gemini integration modules.	Gary	04/11/2026

Contents

Document Change Record 2
Contents 2
1. Design Strategies 3
    1.1. The Planning Phase Before Development 3
    1.2. The Process of Developing 3
    1.3. Correction by Nature 3
        1.3.1. Corrective Maintainability 3
        1.3.2. Preventive Maintainability 3
    1.4. Enhancement by Nature 4
        1.4.1. Adaptive Maintainability 4
        1.4.2. Perfective Maintainability 4
    1.5. Maintainability Practices 4
2. Architectural Design Patterns 5
3. Software Configuration Management Tools 6
    3.1. JIRA 6
    3.2. GitHub 7
    3.3. Google Drive 7
    3.4. Dependency and Environment Management 7

1. Design Strategies

1.1. The Planning Phase Before Development

During the planning phase, the team identified the importance of maintainability, scalability and usability for Fitra, a fitness & health tracking web application designed to help users monitor their fitness & health-related activities and interact with friends. As the application may be expanded with additional health tracking and social features in the future, the system was designed to support modifications and enhancements without requiring extensive changes to existing functionality.

Fitra adopts a modular architecture that separates the frontend, backend and database responsibilities. The frontend is developed using Next.js and Tailwind CSS, while the backend uses FastAPI to handle application logic and API requests. Supabase provides authentication and database services. This separation of concerns allows individual components to be modified, tested and maintained with minimal impact on other parts of the system.

1.2. The Process of Developing

Throughout development, the team follows an iterative approach in which features are implemented, tested and refined incrementally. Each feature, such as user authentication, friend management and exercise planner, is developed as a distinct functional component. This allows defects to be identified and addressed before they affect other parts of the application.

Testing is performed during development to verify that individual features behave as expected and that integrated components work together correctly. For example, the authentication flow is tested to ensure that users can register and log in, while the Friends feature is tested for friend requests, duplicate requests, request handling and friend removal. Feedback from team members and testing results are used to identify usability issues and improve the application's reliability.

1.3. Correction by Nature

Fitra incorporates corrective and preventive maintenance strategies to improve software reliability and reduce the effort required to resolve defects.

1.3.1. Corrective Maintainability

Corrective maintenance focuses on identifying and resolving defects discovered during testing or use. The team tests individual features and their interactions to detect issues such as authentication failures, incorrect friend request handling and database access errors. By investigating and correcting defects at the relevant component, the team can minimise unintended effects on other features.

1.3.2. Preventive Maintainability

Preventive maintenance aims to reduce the likelihood of future defects. Fitra achieves this through modular development, separation of concerns and consistent coding practices. Related functionality is organised into separate pages, components and backend modules, making the code easier to understand and modify. Testing important scenarios, including duplicate friend requests and

database access permissions, also helps identify potential problems before they become more serious.

1.4. Enhancement by Nature

Fitra is designed to accommodate changes in requirements and improvements to existing functionality. Adaptive and perfective maintenance strategies support the application's continued development after its initial release.

1.4.1. Adaptive Maintainability

Adaptive maintenance enables Fitra to accommodate changes in its operating environment, dependencies and external services. The separation between the frontend, FastAPI backend and Supabase services allows changes to one layer to be managed without unnecessarily affecting the others. Configuration through environment variables also allows service endpoints and credentials to be adjusted between development and deployment environments without modifying application logic.

1.4.2. Perfective Maintainability

Perfective maintenance focuses on improving existing features, usability and performance in response to user feedback and changing requirements. Fitra's modular design allows the team to enhance existing functionality or introduce new features, such as additional health tracking capabilities and social interactions, with reduced impact on unrelated components. The team can also refine the user interface and optimise backend operations as testing reveals opportunities for improvement.

1.5. Maintainability Practices

To maintain software quality throughout development, the team adopts the following practices:

- **Readable Code:** Use meaningful variable and function names, consistent formatting and clear code organisation to make the application easier to understand and modify.
- **Version Control:** Use Git and GitHub to track changes, maintain development branches, review contributions and integrate completed features through merges and pull requests.
- **Standardised Documentation:** Document the system architecture, API endpoints, configuration requirements and important implementation decisions to help developers understand and maintain the application.
- **Modularity:** Separate the frontend, backend, authentication, friend management and other functional responsibilities into appropriate components and modules to reduce dependencies between features.
- **Separation of Concerns:** Keep user interface rendering, application logic, authentication and database operations within their appropriate layers to make changes more localised and manageable.
- **Testing:** Test individual features and integration scenarios to identify defects early and verify that changes do not unintentionally break existing functionality.

- **Configuration Management:** Use environment variables for environment-specific settings and service credentials, reducing hardcoded configuration values and simplifying deployment to different environments.

2. Architectural Design Patterns

Fitra uses logical layers within two deployed application services: the Next.js frontend on Node.js and one Python FastAPI backend. The Presentation Layer handles pages, input, navigation and result display. It also maintains draft state and performs MA-5 portion scaling and calorie/macronutrient summation for immediate feedback. These client calculations support review; they do not authorise access, award XP or determine persisted meal totals.

The Application Layer resides in FastAPI and applies authenticated feature rules, request validation, ownership checks and protected persistence. For meal confirmation, it validates reviewed ingredients and delegates transactional saving to Supabase; persisted totals are derived from the ingredient data rather than a client-supplied aggregate. Supabase Auth manages identities, and browser database reads such as friend lookups remain subject to the user's session and RLS.
The Inference and Analytics Layer consists of modules inside the same FastAPI service. Readiness scoring uses the local model with a rule-based fallback, while meal analysis and AI coaching call the external Google Gemini API. These modules can be tested or replaced independently in code; they are not separately deployed application or inference services.

The Data Layer is the single repository of all persistent data, including profiles, activities, check-ins, readiness scores, training plans and meal records.

The Client-Server pattern separates the Next.js interface from the FastAPI REST APIs, alongside Supabase session and RLS-protected access. The Repository pattern isolates training data access. The Strategy pattern provides interchangeable rule-based and machine-learning readiness scorers within FastAPI. Docker Compose provisions Redis, but the active API starts no queue worker and exposes no Strava webhook receiver; current Strava retrieval is request-driven.

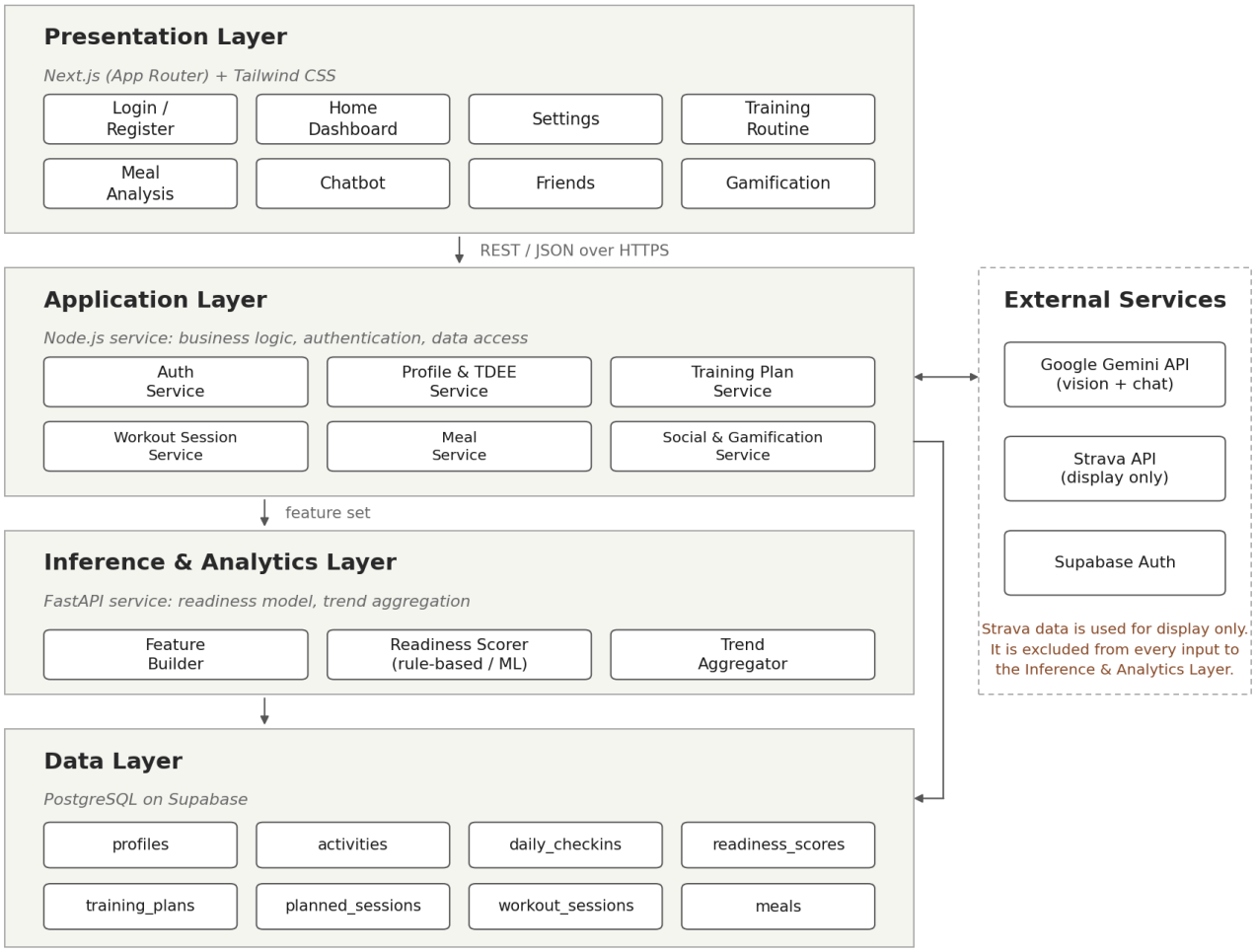


Figure 1: Fitra Architecture design

3. Software Configuration Management Tools

This is where we will discuss version control management and tracking of who made what changes and when.

3.1. JIRA

Jira is used for sprint planning, backlog management and defect tracking. Each sprint backlog is created at the laboratory session, and every task is recorded as a ticket with an owner and a deadline. Defects found during testing are logged as issues, labelled by severity and linked to the requirement or module they affect, so that they are tracked to closure rather than resolved informally. Because Jira records the full history of a ticket, the state of any work item and the reason for a change can be recovered after the fact, which supports both corrective maintenance and the quality assurance reporting required by the project.

3.2. GitHub

GitHub is a source code hosting platform using the distributed version control and source code management system Git. GitHub is chosen for its familiarity and for the support provided by the development environments the team already uses. All source code, database migrations and project documentation are held in the team repository, which serves as the configuration management system for the project. Work is developed on feature branches and merged through pull requests requiring at least one independent approval, so that every change to the main branch has been reviewed and is attributable to an author and a date. GitHub also supports issue tracking similar to a ticketing system: whether it is a software defect, a code enhancement or a documentation change, a team member can open an issue, label it appropriately and assign it to another member to resolve, and all participants receive updates on its progress.

3.3. Google Drive

Google Drive is used as file storage and for the collaborative authoring of project documentation, including the project proposal, the system requirement specification, the quality plan and this report. The service allows team members to edit a document concurrently without a loss of information, and it maintains an automatic version history, so an earlier revision of any document can be restored and the author of any change can be identified. Documents reaching a stable revision are exported and committed to the GitHub repository, so that the released version of each document is held under the same configuration management as the source code.

3.4. Dependency and Environment Management

Dependency versions are themselves placed under configuration management. The Node.js and Python dependency lock files are committed to the repository, so that every team member and every deployment resolves to the same package versions and a defect cannot be caused by two members running different versions of a library. Environment configuration, including external API credentials, is supplied through environment variables and is never committed, while a template file listing the required variables is committed so that a new environment can be reproduced without access to the secrets themselves.